ACTIVIDAD 5: MODELADO DE DATOS CON ESTRUCTURAS (STRUCT) Y PUNTEROS

Laboratorio de Programación (LPR) — 5° 3° A-B

Autores:

Sofia Mojica

Gabriel Zacarias

Mateo Roberts

Leonel De Armas

Docentes a cargo: York Mansilla / Yamil Ganduglia

1. Introducción

El presente informe académico expone el desarrollo de la **Actividad 5**, orientada al modelado de entidades del mundo real aplicadas al Proyecto Integrador Anual (P.I.A.). En el desarrollo de software técnico, la cohesión y organización de los datos en la memoria RAM dictan la eficiencia del sistema. A través del uso de estructuras (`struct`) y punteros en C++, se demuestra cómo agrupar variables heterogéneas en bloques de memoria contiguos y manipularlas mediante pasaje por referencia y sintaxis de desreferenciación indirecta, garantizando la robustez exigida en la Expotécnica 2026.

2. Desarrollo Técnico (Glosario y Análisis del Entorno)

- **Struct (Estructura):** Tipo de dato definido por el usuario que encapsula un conjunto de miembros de diferentes naturalezas tipológicas bajo un mismo bloque lógico de direccionamiento.
- **Operador Flecha (`->`):** Componente sintáctico de C++ utilizado para acceder de forma directa a los miembros de un `struct` a través de un puntero desreferenciado.
- **Pasaje por Referencia:** Técnica que transfiere la dirección de memoria de un objeto a una función, eliminando la necesidad de duplicar datos en el *Stack*.
- **Buffer de Entrada:** Registro intermedio en la memoria RAM donde el sistema operativo retiene los caracteres del teclado antes de ser procesados por los flujos de entrada.
- **`cin.ignore()`:** Función de limpieza que descarta caracteres remanentes (como saltos de línea `\n`) en el flujo de entrada para asegurar lecturas de cadenas continuas.

3. Distribución de Roles y Tareas del equipo

Integrante	Rol / Tarea Asignada
Sofía Mojica	Implementación de estructuras compuestas y lógica del operador flecha
Gabriel Zacarías	Desarrollo de funciones modulares con pasaje por referencia y control del buffer
Mateo Roberts	Estructuración del repositorio, ramas de Git (main/development) y pruebas de compilación
Leonel De Armas	Redacción de manuales técnicos, resolución del cuestionario e informe en APA v7

4. Estructura del Repositorio del Proyecto

sistemaproyecto/
├── .gitignore <-- Reglas de exclusión (.exe, .vscode/)
├── LICENSE <-- Licencia MIT escolar
├── README.md <-- Portada con integrantes y descripción del modelo
├── docs/ <-- Carpeta de documentación formal
│ └── InformeEEST1_LPR2026_ACT05_G03_v1.0.0.pdf <-- ENTREGABLE EN PDF (APA v7)
│ └── manuales/
│ └── manual_programador_v1.0.0.pdf <-- Explicación del struct y operador ->
│ └── manual_programador_v1.0.0.md <-- Explicación del struct y operador ->
├── src/ <-- Código fuente C++
│ └── main.cpp <-- Programa principal modular
└── capturas/ <-- Evidencias de ejecución (.png)
└── ejecucion_struct.png

5. Desarrollo de los Cuestionarios Técnicos de Control

- **Heterogeneidad: ¿Qué ventaja ofrece agrupar los datos de un sensor o dispositivo en un struct en lugar de declarar muchas variables sueltas?**
  Respuesta: Ofrece cohesión de datos y legibilidad arquitectónica. Al agrupar variables dispares (como `int id`, `float lectura` y `char nombre`) en un `struct`, el compilador reserva un bloque de memoria unificado y contiguo. Esto evita tener datos huérfanos dispersos en la RAM, permite manipular el dispositivo completo como un único objeto y facilita pasar la entidad a cualquier función modular con un solo parámetro, emulando abstracciones avanzadas de bajo nivel.
- **Operador Flecha: ¿Por qué el compilador de C++ no permite usar la notación de punto (ptr.id) cuando la variable es de tipo puntero?**
  Respuesta: El operador punto (`.`) está diseñado de forma rígida para operar únicamente sobre instancias directas de una estructura alojadas en la pila. Cuando se trabaja con un puntero, el valor de la variable es una dirección hexadecimal de memoria, no el objeto físico. Para emplear la notación de punto se requeriría desreferenciar primero el puntero de forma manual escribiendo `(*ptr).id`. C++ introduce el operador flecha (`->`) como un azúcar sintáctico para fusionar la desreferenciación y el acceso al miembro en una única instrucción de código.
- **Eficiencia en RAM: ¿Por qué el pasaje por referencia mediante punteros optimiza la memoria procesada frente al pasaje por copia?**
  Respuesta: El pasaje por copia obliga a la CPU a duplicar todos los valores de los miembros de la estructura en un nuevo segmento de la pila (*Stack*), lo que genera un consumo innecesario de memoria y desperdicia ciclos de reloj

copiando datos. En contraposición, el pasaje por referencia transfiere únicamente un puntero con la dirección hexadecimal (un valor fijo de 4 u 8 bytes según la arquitectura del procesador), manipulando los datos en su origen sin generar redundancia en la RAM.

- **Buffer de Entrada: ¿Qué comportamiento erróneo ocurre en la terminal si se omite la instrucción cin.ignore() antes de un getline?**
  Respuesta: Al procesar una entrada numérica con `cin >>`, el usuario digita el valor y presiona la tecla Enter. El flujo extrae el número, pero deja el carácter del salto de línea (`\n`) atrapado de forma remanente en el buffer de entrada. Si a continuación se invoca un `cin.getline()`, esta función examina el buffer, detecta el `\n` remanente e interpreta que el usuario envió una cadena vacía, provocando que la terminal "se salte" la lectura de texto y rompa la secuencia lógica de ejecución.
- **Aplicación al Proyecto Integrador: ¿Qué entidad o componente hardware de su Proyecto Integrador Anual representaron con esta estructura y qué variables componen sus miembros?**
  Respuesta: Para nuestro Proyecto Integrador Anual enfocado en un sistema de automatización, modelamos la entidad **ModuloHardware** dentro del archivo de código. El modelo quedó definido con los miembros: `int id` para registrar el canal del puerto físico, `char nombre[50]` para almacenar la etiqueta identificadora del componente en el sistema y `float metrica` para resguardar los valores decimales de las variables de sensado continuo procesadas por el firmware.

6. Conclusión

El modelado de datos mediante estructuras sólidas e instrucciones orientadas a punteros representa un hito fundamental en la programación técnica estructurada. La implementación exitosa de la metodología *Top-Down* permite fragmentar problemas de hardware complejos en bloques de código acotados y reutilizables. Controlar los flujos de la memoria RAM mediante des-referenciaciones indirectas (`->`), pasajes por dirección (`&`) y limpiezas minuciosas de los buffers de hardware dota al equipo de desarrollo de las competencias mecánicas esenciales para programar sistemas embebidos eficientes y profesionales de cara al cierre de las evaluaciones de la materia LPR.

7. Enlaces y Aportes Individuales de Integrantes

Cada integrante del grupo debe ingresar a su apartado correspondiente a continuación y colocar su link individual (repositorio de GitHub).

1. Gabriel Zacarias

https://github.com/GabZac/Actividad-5-Modelado-de-datos-con-estructuras-y-Punteros.git

2. Sofia Mojica

   https://github.com/Ailuna77/ACTIVIDAD-5-MODELADO-DE-DATOS-CON-ESTRUCTURAS-STRUCT-Y-PUNTEROS.git